In [3]:
```
# is have only 2 feautuers
import joblib
from clean_text import clean_text
import pandas as pd
from clean_lemmitezer import  after_cleaning_text



model = joblib.load("sentiment_model.pkl")
tfidf = joblib.load("tfidf_vectorizer.pkl")
# all_stopwords = joblib.load("stopwords.pkl")



text =['thanks teflon liver scottish raised whisky']


# #
# from nltk.corpus import wordnet
# from nltk import pos_tag

# def get_wordnet_pos(tag):
#     if tag.startswith('J'):
#         return wordnet.ADJ
#     elif tag.startswith('V'):
#         return wordnet.VERB
#     elif tag.startswith('N'):
#         return wordnet.NOUN
#     elif tag.startswith('R'):
#         return wordnet.ADV
#     else:
#         return wordnet.NOUN



#   #it can reduce words correctly like runnig to run .
# from nltk.stem import WordNetLemmatizer
# lemmatizer = WordNetLemmatizer()

# #stopwords_set = build_stopwords(text)


# def after_cleaning_text(text):
#     words = text.split()
#     tagged_words = pos_tag(words)

#     cleaned_words = []



#     for w, pos in tagged_words:
#         if w in all_stopwords:#add my auto remove after visulization stop words
#             continue
```

```
#         wn_pos = get_wordnet_pos(pos)

#         lemma = lemmatizer.lemmatize(w, wn_pos)

#         # 🔥 fix for verbs (important)
#         if wn_pos == 'v' and lemma == w:
#             lemma = lemmatizer.lemmatize(w, 'v')

#         cleaned_words.append(lemma)

#     return " ".join(cleaned_words)
# #

samples_clean = [clean_text(s) for s in text]

lemmatized_text = pd.Series(samples_clean).apply(after_cleaning_text)

X_sample = tfidf.transform(lemmatized_text)

preds = model.predict(X_sample)

label_map = {
    0: "Negative 😡",
    1: "Neutral 😐",
    2: "Positive 😊"
}

for text, pred in zip(text, preds):
    print(text, "->", label_map[pred])
```

```
thanks teflon liver scottish raised whisky -> Positive 😊
```

This project is a simple Natural Language Processing (NLP) based sentiment analysis system. The main goal of this project is to classify input text into three categories: Positive, Negative, and Neutral. The model is built using machine learning techniques with TF-IDF vectorization for feature extraction and a trained classification model for prediction.

The workflow starts with taking raw text input from the user. The text is first cleaned using basic preprocessing techniques such as lowercasing, removing unwanted characters, and normalizing the text. After that, lemmatization is applied using POS tagging to reduce words to their root form, which helps improve model accuracy by reducing noise in the data.

Once the text is preprocessed, it is converted into numerical form using a TF-IDF vectorizer that transforms the text into meaningful features. These features are then passed into a trained machine learning model, which predicts the sentiment of the given text.

Finally, the output is displayed as either Positive, Negative, or Neutral based on the model's prediction. This project demonstrates a complete NLP pipeline including data cleaning, feature engineering, and machine learning prediction in a simple and practical way.